Appendix B 練習

作答時標示 text/data base、location counter、alignment、instruction width、relocation type 與 PC 定義。Pseudoinstruction 必須展開後再計算。

GB-Q01 Two-pass label addresses

簡化 MIPS object 的 `.text` base 是 `0x00400000`，`.data` base 是 `0x10010000`。每條 instruction 4 bytes。

```
.text
main:  addiu $t0,$zero,0
loop:  addiu $t0,$t0,1
       beq   $t0,$a0,loop
       nop
done:  jr    $ra
       nop
func:  addiu $v0,$zero,7
       jr    $ra
       nop
.data
.align 2
table: .word 0xdeadbeef
count: .word 3
str:   .asciiz "Hi"
```

1. 列出 `main`、`loop`、`done`、`func`、`table`、`count`、`str` 的 addresses。
2. `beq` 位於哪個 address？其 PC-relative immediate 應是多少？
3. 假設要另行編碼 `j done` 與 `jal func`，兩者的 26-bit target fields 分別是多少？兩條假設指令不插入上列程式，不改變標籤位址。
4. `.data` 最後一個 byte 的 address 是什麼？

GB-Q02 Relocations

Object A 的 `.text` 被 linker 放在 `0x00401000`。Offset `0x10` 的 instruction 是 `jal external`，linker 將 external text symbol 定在 `0x00402000`。

另一段程式要把 absolute address `0x12345678` 載入 register，使用 `lui` high 與 `ori` low pair。此處 `ori` 將 immediate 零延伸，不採 signed-low adjustment；請另外說明若改用 `lui/addiu` 時 high half 如何調整。

Appendix B 練習 （續）

1. `jal external` 最終 26-bit target field 是多少？
2. 完整 J-format word 的 opcode `jal=3` 加上 target field 後的 hex 是什麼？
3. `lui` immediate 與 `ori` immediate 應是多少？
4. 若 assembler 把 relocation 留到 linker，object 中的 field 與 relocation metadata 各扮演什麼角色？

GB-Q03 Linker merge

Object P 有 192 bytes `.text`、16 bytes `.data`；Object Q 有 64 bytes `.text`、24 bytes `.data`。Text section 從 `0x00400000` 開始，Data 從 `0x10010000` 開始。Section 本身按 4-byte alignment；Q 的 data 起始另要求 8-byte alignment。

1. P、Q 的 text start addresses 是什麼？
2. 不考慮額外 requirement 時，P、Q data start addresses為何？
3. 套用 Q data 的 8-byte alignment後，Q data 的 start 與 final data end 在哪裡？
4. 若 P 和 Q 都定義 global symbol `value`，linker 能否自行任選一個？說明 resolution/error 原則。

GB-Q04 Stack frame 與 recursion

本題採簡化教學frame，不宣稱完整o32 outgoing-argument area；`sum_to(n)`每層frame 16 bytes：`ra` 在 offset 12、`a0` 在 offset 8，其餘保留 alignment/local。Base case $n = 0$ 也建立同樣 frame。函式 recursive call 深度包含 base case。

1. 呼叫 `sum_to(3)` 時，最多同時存在幾層 frames，共多少 stack bytes？
2. 最內層的 `$ra` 與 `$a0` 保存在哪些 relative offsets？其 absolute offsets 如何由 current `$sp` 表示？
3. Return 時 frame 以什麼順序 unwind？若先執行 `addiu $sp,$sp,16`，為何不能繼續使用原本的 `12($sp)` 讀取 saved `$ra`？
4. Interrupt 在 recursive call 中發生時，application frame 與 exception handler context 的保存責任有何不同？

GB-Q05 Endianness 與 alignment

32-bit value `0xdeadbeef` 從 byte address `0x100` 存入。

Appendix B 練習 （續）

1. Big-endian 與 little-endian 各自 bytes at `0x100..0x103` 是什麼？
2. 若以 `lw` 從 `0x102` 讀 word，在 strict-alignment MIPS 會怎樣？
3. `0xdeadbeef` 的一半 `0xbeef` 存在 `0x100` 起，並用 big-endian `lhu` 讀取，value 是多少？
4. 設計 network packet 時，為何不能只靠 host memory dump 判斷 wire format？

GB-Q06 Exception/syscall state

使用者程式執行 SPIM-style `syscall`。本題以概念性MIPS exception handler模型處理print-int，不宣稱SPIM內部真的執行這份handler程式。明定syscall不在delay slot，Cause.BD=0、saved EPC為syscall本身位址；服務完成後返回。

1. Handler 需要讀哪些 registers 以取得 service number、argument 與回傳 result？依一般 teaching convention 命名。
2. 為何 syscall handler return 前通常把 saved EPC 加 4？
3. Arithmetic overflow exception 也能直接把 EPC 加 4 後返回嗎？說明差異。
4. Handler 進入時為何要保存它會 clobber 的 registers？若不保存且 handler nested，可能發生什麼？

GB-Q07 Pseudoinstruction 與 SPIM I/O

對 `move $s0,$s1` 與 `la $t0,label`：

1. 分別寫出一種可能的 machine-instruction expansion。
2. `la` 的 expansion 為何可能依 label 的 final address/position independence 而不同？
3. 在 SPIM teaching convention 中，若要印 null-terminated string，`$v0` 與 `$a0` 分別應放什麼？
4. 若要印 integer 後終止 process，列出 service numbers 與 argument register安排。為何不能只看 assembler source 斷定 binary size？

GB-Q08 Section 合併與 relocation 的聯合追蹤

P.text長0x24 bytes、Q.text長0x14；Q.text需16-byte alignment。Text base0x00401000。P.data長5 bytes、Q.data長4且需8-byte alignment，data base0x10010000。Q的foo在Q.text offset4，item在Q.data offset0。P.text offset8是jal foo，offset0x10/0x14是一對lui/addiu，用於載入item。只採講義的明確S+A最終值模型，addend A=0。

Appendix B 練習 （續）

1. 求Q的text/data bases及兩段padding。
2. 求foo、item位址；求jal field、完整word與ra。
3. 求lui/addiu high及low；若item改定址為0x1001fffc，兩個immediates如何改變？
4. 為什麼不能把跨區域的jal直接截26 bits？

GB-Q09 Archive 選取與循環相依

沿用講義由左至右、每個archive內反覆抽取至穩定、離開後不重掃的模型。main.o引用f。libA的A1.o定義f、引用g，A2.o定義h；libB的B1.o定義g、引用h。

1. 追蹤main.o libA libB，列每步納入的members及未解symbol集合。
2. 若再列一次libA，為何可以成功？
3. 若把A1/A2合成同一個object member，原始順序是否仍失敗？解釋archive抽取的最小單位。

GB-Q10 遞迴正確性與 ABI 反例

使用講義sum_to，初始sp=0x2000、n=2，base case也建立frame，stack空間足夠。

1. 列每層sp、saved n、saved ra地址及返回v0。
2. 若把lw t0,8(sp)移到jal之前，並刪掉原位置的load，為何一般callee-saved規則不能保證正確？用子呼叫改t0的情形說明。
3. 若把addu v0,v0,t0放到jal的delay slot，為何也錯？
4. 給出輸入與stack的必要前提，並以歸納重述此函式的契約。

GB-Q11 EPC 與 MMIO 的不同控制流程

1. Branch在0x00400040、slot的lw在0x00400044，slot發生可恢復page fault。求BD、EPC；為什麼不能把EPC固定加4當作所有例外返回？
2. 另一個BD=0的syscall在0x00400080，handler已完整完成服務。明定handler需手動選擇下一PC，求順序返回位址。與第1小題的重試需求比較。
3. 講義receiver control依序0、0、1，data為0x141。求control/data loads次數與最後v0。若把data load放在beq delay slot，哪個裝置不變量被破壞？

GB-Q12 遠距分支與 delay slot

採講義的 pre-Release-6 MIPS32 模型。`beq $s0,$s1,far` 位於 0x00400000，其 delay slot 是 `nop`；`far=0x00430000`。

Appendix B 練習 （續）

1. 算出原分支的位移，判定 signed 16-bit 欄位能否表示。
2. 用反向條件分支及 `j` 改寫，明示兩個 delay slots 和不相等時的後續位置。
3. 對相等與不相等分別追蹤控制流程，算出反向分支的 immediate；說明 `j` 還有什麼可達範圍限制。

參考資料與署名

David A. Patterson、John L. Hennessy，Computer Organization and Design，Revised Fourth Edition；Lin Cing，《計算機組織》手寫筆記；計算機組織與結構（CSO）課程投影片。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study